



PASHUPATASTRA

AI-Assisted Railway Maintenance Scheduling & Disruption Recovery

Document: Formal Domain Review & Complete Master Data Package

Branch: `feature/domain-data`

Event: Smart India Hackathon 2026

Workstream: Domain Modeling & Synthetic Data (Darshini)

Integration Owner: Aryan (`feature/backend-api`)

Status: Ready for Team Domain Freeze Checkpoint

PART I: FORMAL DOMAIN REVIEW (DOMAIN_REVIEW.md)

Executive Summary

Pashupatastra solves a critical real-world operational problem for Indian Railways (IR): **how to optimize, schedule, and dynamically re-schedule maintenance blocks (possessions) across busy railway corridors while respecting operational safety, asset conditions, precedence dependencies, and train timetables.**

This domain review formalizes the railway domain semantics, identifies limitations in initial baseline assumptions, and establishes concrete recommendations for the **Domain Freeze** checkpoint before downstream workstreams (Tyagi's Optimizer, Ayush's ML Scorer, Tirth's Simulation, Archit's Frontend, and Aryan's API) build further.

1. Track Modeling & Corridor Representation

Current Assumption: Tracks are identified by simple flat strings: `track_id = "UP-1"` , `track_id = "DOWN-1"` .

Problem Identified: A flat string alone does not convey which corridor, section, or kilometer range the track belongs to. However, introducing a full GIS graph or topological switch network adds unnecessary mathematical complexity to the CP-SAT solver without adding demo value.

Recommendation: **Retain `track_id: str` as the primary key/unique identifier** in shared contracts (e.g. `"UP-1"` , `"DOWN-1"`). In the Domain Data layer, represent tracks with structured metadata (corridor ID, section name, direction, mileage start/end, speed limit, electrification).

Reason & Grounding: In Indian Railways, maintenance blocks are granted per **block section** along a directional line (Up Main Line, Down Main Line). Preserving a clean string identifier keeps CP-SAT interval constraints simple (no overlapping intervals on the same track), while metadata enables rich UI display on the Frontend dashboard.

Downstream Impact: **Optimizer (Tyagi):** Zero breaking changes. **Frontend (Archit):** Groups tracks cleanly by direction. **ML Scorer (Ayush):** Uses speed limits and density for operational impact scoring.

2. Maintenance Work Types (`WorkType`)

Current Assumption: Generic categories (`RENEWAL` , `INSPECTION` , `REPAIR` , `MAINTENANCE`).

Problem Identified: Categories are too generic and fail to reflect Indian Railways maintenance practices (e.g. Track Machine Operations, OHE Power Blocks, S&T Interlocking). They do not capture why certain tasks require heavy machinery or complete electrical isolation.

Recommendation: Standardize the WorkType enum into 6 concrete railway maintenance categories:

WorkType	Description	Typical Duration	Machinery / Resource Note	Priority Range
TRACK_RENEWAL	Complete Track Renewal (CTR), Through Sleeper Renewal (TSR), Rail Renewal (TRR)	180 – 360 min	Heavy possession (BCM, PQRS, T-28), total track closure	0.85 – 0.95
BALLAST_TAMPING	Track packing and alignment using Tie-Tamping machines (CSM, Duomatic)	90 – 180 min	Requires safety headway buffer after track renewal	0.70 – 0.85
OHE_MAINTENANCE	Overhead Equipment inspection, contact wire replacement, insulator washing	60 – 150 min	Requires Electrical Power Block (OHE isolation); Tower Wagon	0.65 – 0.80
SIGNALLING_INTERLOCKING	Point machine overhaul, track circuit testing, axle counter maintenance	45 – 120 min	Affects interlocking & station approaches; Signal Gang	0.60 – 0.75
ROUTINE_INSPECTION	Ultrasonic Flaw Detection (USFD), foot patrol, geometry recording	30 – 90 min	Minimal disruption; manual trolley or inspection unit	0.35 – 0.50
EMERGENCY_REPAIR	Rail fracture, weld failure, OHE wire parting, point failure	60 – 180 min	Unscheduled, pre-emptive, highest urgency score	0.95 – 0.99

3. Disruption Types (`DisruptionType`)

We define 4 standardized, deterministic disruption types for the **PLAN** → **DISRUPT** → **RECOVER** demonstration:

1. `TRACK_UNAVAILABLE` (Scenario 1 — Track Blockage)

Description: A track segment becomes unusable from time t_1 to t_2 (e.g. detected rail defect, obstacle, speed restriction).

Impact: Blocks scheduled on that track during that interval become `AFFECTED` and must be rescheduled to another track or moved in time.

2. `EMERGENCY_WORK` (Scenario 2 — Critical Unscheduled Work Insertion)

Description: An urgent emergency repair block (e.g. rail fracture) appears at time t with maximum priority (0.99).

Impact: CP-SAT must schedule the emergency block immediately, shifting or dropping lower-priority routine blocks while protecting already committed blocks.

3. `POSSESSION_CURTAILMENT` (Scenario 3 — Time Window Shortened)

Description: Due to delayed express trains, an available possession window (e.g. 08:00–12:00) is curtailed to (09:30–11:30).

Impact: Maintenance tasks that no longer fit must be compressed, rescheduled to later slots, or marked unscheduled with clear reason codes.

4. `ASSET_BREAKDOWN` (Scenario 4 — Machine / Equipment Failure)

Description: A heavy track machine (e.g. BCM) breaks down mid-block, extending work duration or halting all tasks sharing that machine group.

4. Resource & Machinery Constraints

Current Assumption: Complex multi-resource models (`crew_id` , `crew_type` , `equipment_id` , `crew_capacity`).

Problem Identified: Full resource leveling and human crew rostering turns the problem into a 2-stage multi-mode RCPSP, risking slow solve times (> 10s) and artificial infeasibility during a live hackathon demo.

Recommendation: **Do NOT model individual human workers in Phase 1.** Model shared heavy machines via `mutual_exclusion_group: Optional[str]` (e.g. `BCM_MACHINE_01` , `CSM_TAMPER_01` , `TOWER_WAGON_01`). The CP-SAT solver enforces:
$$\text{Interval}(\text{Block}_1) \cap \text{Interval}(\text{Block}_2) = \emptyset \quad \text{for all blocks sharing same machine group}$$
 This guarantees that the single available BCM machine is not scheduled in two places simultaneously, without bloating the solver.

5. Train Timetable & Operational Window Representation

Current Assumption: `train_timetable` field exists in the contract but lacks clear solver constraints.

Recommendation: Model train traffic as **Discrete Available Possession Windows** per track over the 24-hour horizon:

Window ID	Track	Window Type	Clock Time	Minute Timeline	Allocated Maintenance Work
POS-UP1-NIGHT	UP-1	NIGHT_TRAFFIC_BLOCK	00:30 – 05:00	0030 – 0300 (270 min)	Heavy Track Renewal (CTR) & Ballast Tamping
POS-UP1-MIDDAY	UP-1	MIDDAY_MAINTENANCE	11:30 – 14:30	0690 – 0870 (180 min)	USFD Inspection & OHE Isolations
POS-UP1-EVENING	UP-1	EVENING_OFF_PEAK	21:00 – 23:30	1260 – 1410 (150 min)	Point Machines & Signalling Interlocking
POS-DN1-NIGHT	DOWN-1	NIGHT_TRAFFIC_BLOCK	00:30 – 05:00	0030 – 0300 (270 min)	Heavy Track Renewal (CTR) & Ballast Tamping
POS-DN1-MIDDAY	DOWN-1	MIDDAY_MAINTENANCE	11:30 – 14:30	0690 – 0870 (180 min)	USFD Inspection & OHE Isolations
POS-DN1-EVENING	DOWN-1	EVENING_OFF_PEAK	21:00 – 23:30	1260 – 1410 (150 min)	Point Machines & Signalling Interlocking

Solver Constraint: Every scheduled block $[s_i, e_i]$ on track T must lie strictly within an available window:

$$\exists W \in \text{PossessionWindows}(T) \quad \text{such that} \quad W.\text{start} \leq s_i \quad \text{and} \quad e_i \leq W.\text{end}$$

6. Precedence Dependencies & Sequential Chains

Maintenance operations follow strict sequential engineering protocols:

$$\text{Track Renewal (CTR)} \rightarrow \text{Deep Ballast Screening (BCM)} \rightarrow \text{Tie Tamping (CSM)} \rightarrow \text{Speed Normalization}$$

If Block B depends on Block A (dependencies: `["BLK-A"]`), CP-SAT enforces: $\text{start_time}(B) \geq \text{end_time}(A) + \text{buffer}$.

7. Summary Checklist for Domain Freeze

- ✔ **Track Modeling:** String `track_id` preserved for solver; metadata dictionary available for UI.
- ✔ **Work Types:** 6 standardized Indian Railways categories agreed.
- ✔ **Disruption Types:** 4 deterministic disruption types defined.
- ✔ **Resources:** Simplified to `mutual_exclusion_group` for heavy machines.
- ✔ **Train Timetable:** Modeled as discrete `PossessionWindow` intervals.
- ✔ **Synthetic Generator:** Seed-driven implementation ready for downstream integration.

PART II: FULL SYNTHETIC DATASETS & SCENARIOS

Scenario A: Baseline Mainline Corridor (`corridor_a_blocks.json`)

Corridor: `CORRIDOR_A` (New Delhi – Ghaziabad Main Line) | **Tracks:** `UP-1` , `DOWN-1` | **Horizon:** 1440 min | **Candidates:** 12 Blocks

Block ID	Track	Asset ID	Work Category	Dur	Pri	Risk	Time Window (Clock)	Machine Group	Precedence
BLK-001	DOWN-1	AST-DOWN-1-OHE-013	OHE_MAINTENANCE	120 m	0.79	0.63	11:00 – 15:30 (0660-0930)	TOWER_WAGON_01	—
BLK-002	DOWN-1	AST-DOWN-1-RAI-015	BALLAST_TAMPING	105 m	0.50	0.50	00:30 – 06:00 (0030-0360)	CSM_TAMPER_01	—
BLK-003	DOWN-1	AST-DOWN-1-RAI-014	TRACK_RENEWAL	210 m	0.59	0.66	00:30 – 06:00 (0030-0360)	BCM_MACHINE_01	—
BLK-004	DOWN-1	AST-DOWN-1-RAI-009	BALLAST_TAMPING	150 m	0.59	0.59	04:00 – 07:30 (0240-0450)	CSM_TAMPER_01	BLK-003 
BLK-005	UP-1	AST-UP-1-OHE-004	OHE_MAINTENANCE	75 m	0.56	0.52	00:30 – 23:30 (0030-1410)	TOWER_WAGON_01	—
BLK-006	DOWN-1	AST-DOWN-1-OHE-016	OHE_MAINTENANCE	120 m	0.54	0.49	00:30 – 23:30 (0030-1410)	TOWER_WAGON_01	—
BLK-007	DOWN-1	AST-DOWN-1-OHE-011	OHE_MAINTENANCE	120 m	0.79	0.53	11:00 – 15:30 (0660-0930)	TOWER_WAGON_01	—
BLK-008	UP-1	AST-UP-1-RAI-001	TRACK_RENEWAL	210 m	0.78	0.77	00:30 – 06:00 (0030-0360)	BCM_MACHINE_01	—
BLK-009	UP-1	AST-UP-1-TUR-007	SIGNALLING_INTERLOCK	105 m	0.42	0.44	00:30 – 23:30 (0030-1410)	S_T_GANG_01	—
BLK-010	UP-1	AST-UP-1-RAI-008	ROUTINE_INSPECTION	45 m	0.42	0.46	11:00 – 15:30 (0660-0930)	—	—

Block ID	Track	Asset ID	Work Category	Dur	Pri	Risk	Time Window (Clock)	Machine Group	Precedence
BLK-011	UP-1	AST-UP-1-SIG-002	SIGNALLING_INTERLOCK	75 m	0.65	0.59	00:30 – 23:30 (0030-1410)	S_T_GANG_01	—
BLK-012	UP-1	AST-UP-1-TUR-006	SIGNALLING_INTERLOCK	75 m	0.57	0.50	00:30 – 23:30 (0030-1410)	S_T_GANG_01	—

Scenario B: Dense Quadruple Corridor (corridor_b_dense.json)

Corridor: CORRIDOR_B_DENSE | **Tracks:** UP-1 , UP-2 , DOWN-1 , DOWN-2 | **Candidates:** 24 Blocks

Simulates heavy quadruple trunk routes (e.g. New Delhi – Kanpur). Contains 4 competing track machine groups (BCM_MACHINE_01 , CSM_TAMPER_01 , TOWER_WAGON_01 , S_T_GANG_01) with tight possession windows to test CP-SAT solver optimization trade-offs under high network saturation.

Scenario C: Disruption & Recovery Testbed (corridor_c_disrupted.json)

Corridor: CORRIDOR_C_DISRUPTED | **Tracks:** UP-1 , DOWN-1 | **Candidates:** 14 Blocks

Pre-Committed Blocks (Locked by Section Controller):

- BLK-001 : DOWN-1 , 01:00 – 03:00 (0060–0180 min), **COMMITTED** (Cannot be shifted).
- BLK-002 : DOWN-1 , 01:30 – 03:15 (0090–0195 min), **COMMITTED** (Cannot be shifted).

Disruption Event: TRACK_UNAVAILABLE on UP-1 from 02:00 to 05:00. Tirth's simulation will verify that Tyagi's optimizer re-routes unaffected blocks while protecting BLK-001 & BLK-002 .

Automated Test Verification Results

```
test_dependency_integrity (test_generator.TestCorridorDataGenerator) ..... OK
test_different_seeds_produce_variation (test_generator.TestCorridorDataGenerator) OK
test_json_fixtures_validity (test_generator.TestCorridorDataGenerator) ..... OK
test_scenario_b_dense_properties (test_generator.TestCorridorDataGenerator) ..... OK
test_scenario_c_committed_blocks (test_generator.TestCorridorDataGenerator) ..... OK
test_score_boundaries (test_generator.TestCorridorDataGenerator) ..... OK
test_seed_reproducibility (test_generator.TestCorridorDataGenerator) ..... OK
test_time_window_consistency (test_generator.TestCorridorDataGenerator) ..... OK
test_track_assignment_validity (test_generator.TestCorridorDataGenerator) ..... OK
test_work_types_are_valid_enums (test_generator.TestCorridorDataGenerator) ..... OK

-----
Ran 10 tests in 0.049s – OK (100% Passed)
```